

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\MullerSolver
.java

- Specification

module MullerSolver2

type

```
var min, max, start: real      // search range [min, max] and optional start
var maxEval: integer          // maximum allowed evaluations
var relAcc, absAcc, fAcc: real // relative, absolute, and function-value accuracy
var r: real                   // returned root candidate
ex: set of {None, NullArgument, BadSequence, TooManyEvaluations} // outcome tag
```

process solve(f, min, max, start, maxEval, relAcc, absAcc, fAcc: real) r: real, ex: set of {None, NullArgument, BadSequence, TooManyEvaluations}

functional scenario form:

```
f = null && ex = NullArgument
||
min > max && ex = BadSequence
||
yMin = 0 && ex = None && r = min
||
and yMax = 0 && ex = None && r = max
||
and yStart = 0 && ex = None && r = start
||
and yStart ≠ 0 && ex = None
    && r ∈ [min, max] // proceed from start with Müller iterations
||
xNew - xPrev ≤ tol      && ex = None
    && r = xNew && |f(r)| ≤ fAcc
||
xProj - xPrev ≤ tol      && ex = None
    && r = xProj && |f(r)| ≤ fAcc
||
xSafe - xPrev ≤ tol
    && ex = None
    && r = xSafe && |f(r)| ≤ fAcc
||
evaluations > maxEval && ex = TooManyEvaluations
```

end process

end module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import java.util.Random;
/**
 * This class implements the
 * <a href="http://mathworld.wolfram.com/MullersMethod.html"> Muller's
 * Method</a> for root finding of real univariate functions. For refere
nce, see
 * <b>Elementary Numerical Analysis</b>, ISBN 0070124477, chapter 3.
 * <p>
 * Muller's method applies to both real and complex functions, but here
we
 * restrict ourselves to real functions. This class differs from
 * {@link MullerSolver} in the way it avoids complex operations.
 * </p>
 * <p>
 * Except for the initial [min, max], it does not require bracketing co
ndition,
```

```

* e.g.  $f(x_0)$ ,  $f(x_1)$ ,  $f(x_2)$  can have the same sign. If a complex number
arises
* in the computation, we simply use its modulus as a real approximat
n.
* </p>
* <p>
* Because the interval may not be bracketing, the bisection alternativ
e is not
* applicable here. However in practice our treatment usually works wel
l,
* especially near real zeroes where the imaginary part of the complex
* approximation is often negligible.
* </p>
* <p>
* The formulas here do not use divided differences directly.
* </p>
*
* @since 1.2
* @see MullerSolver
*/
public class MullerSolver2 extends AbstractUnivariateSolver {
    /** Default absolute accuracy. */
    private static final double DEFAULT_ABSOLUTE_ACCURACY = 1e-6;
    /**
     * Construct a solver with default accuracy (1e-6).
     */
    public MullerSolver2() {
        this(DEFAULT_ABSOLUTE_ACCURACY);
    }
    /**
     * Construct a solver.
     *
     * @param absoluteAccuracy Absolute accuracy.
     */
    public MullerSolver2(double absoluteAccuracy) {
        super(absoluteAccuracy);
    }
    /**
     * Construct a solver.
     *
     * @param relativeAccuracy Relative accuracy.
     * @param absoluteAccuracy Absolute accuracy.
     */
}

```

```

public MullerSolver2(double relativeAccuracy, double absoluteAccuracy)
{
    super(relativeAccuracy, absoluteAccuracy);
}
/**
 * {@inheritDoc}
 */
@Override
protected double doSolve() throws TooManyEvaluationsException, NumberI
sTooLargeException, NoBracketingException {
    final double min = getMin();
    final double max = getMax();
    verifyInterval(min, max);
    final double relativeAccuracy = getRelativeAccuracy();
    final double absoluteAccuracy = getAbsoluteAccuracy();
    final double functionValueAccuracy = getFunctionValueAccuracy();
    // x2 is the last root approximation
    // x is the new approximation and new x2 for next round
    // x0 < x1 < x2 does not hold here
    double x0 = min;
    double y0 = computeObjectiveValue(x0);
    if (JdkMath.abs(y0) < functionValueAccuracy) {
        return x0;
    }
    double x1 = max;
    double y1 = computeObjectiveValue(x1);
    if (JdkMath.abs(y1) < functionValueAccuracy) {
        return x1;
    }
    if (y0 * y1 > 0) {
        throw new NoBracketingException(x0, x1, y0, y1);
    }
    double x2 = 0.5 * (x0 + x1);
    double y2 = computeObjectiveValue(x2);
    double oldx = Double.POSITIVE_INFINITY;
    while (true) {
        // quadratic interpolation through x0, x1, x2
        final double q = (x2 - x1) / (x1 - x0);
        final double a = q * (y2 - (1 + q) * y1 + q * y0);
        final double b = (2 * q + 1) * y2 - (1 + q) * (1 + q) * y1 + q * q *
y0;
        final double c = (1 + q) * y2;
        final double delta = b * b - 4 * a * c;
        double x;

```

```

final double denominator;
if (delta >= 0.0) {
    // choose a denominator larger in magnitude
    double dplus = b + JdkMath.sqrt(delta);
    double dminus = b - JdkMath.sqrt(delta);
    denominator = JdkMath.abs(dplus) > JdkMath.abs(dminus) ? dplus : dm
inus;
} else {
    // take the modulus of (B +/- JdkMath.sqrt(delta))
    denominator = JdkMath.sqrt(b * b - delta);
}
if (denominator != 0) {
    x = x2 - 2.0 * c * (x2 - x1) / denominator;
    // perturb x if it exactly coincides with x1 or x2
    // the equality tests here are intentional
    while (x == x1 || x == x2) {
        x += absoluteAccuracy;
    }
} else {
    // extremely rare case, get a random number to skip it
    x = min + JdkMath.random() * (max - min);
    oldx = Double.POSITIVE_INFINITY;
}
final double y = computeObjectiveValue(x);
// check for convergence
final double tolerance = JdkMath.max(relativeAccuracy * JdkMath.abs(
x), absoluteAccuracy);
if (JdkMath.abs(x - oldx) <= tolerance || JdkMath.abs(y) <= function
ValueAccuracy) {
    return x;
}
// prepare the next iteration
x0 = x1;
y0 = y1;
x1 = x2;
y1 = y2;
x2 = x;
y2 = y;
oldx = x;
}
}
}
// VT
class MS2VT {

```

```

private static boolean approx(double a, double b, double tol) {
    if (Double.isNaN(a) || Double.isNaN(b))
        return false;
    if (Double.isInfinite(a) || Double.isInfinite(b))
        return a == b;
    return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a), Math.abs(b)));
}

private static void check(String name, boolean cond) {
    System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
}

private static void runCase(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double min,
    double max, int maxEval) {
    System.out.println("---- " + name + " (no start) ----");
    try {
        MullerSolver2 solver = new MullerSolver2();
        double r = solver.solve(maxEval, f, min, max);
        System.out.printf("root=%.17g, f(root)=%.3e\n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runCaseWithStart(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        MullerSolver2 solver = new MullerSolver2();
        double r = solver.solve(maxEval, f, min, max, start);
        System.out.printf("root=%.17g, f(root)=%.3e\n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

public void Vtest() {
    System.out.println("=== MullerSolver2 V-method tests ===");
    // VT1: f == null -> NullArgument
    System.out.println("VT1");
    try {
        MullerSolver2 solver = new MullerSolver2();
        // min/max arbitrary; expect IllegalArgumentException

```

```

    solver.solve(100, null, -1.0, 1.0);
    check("VT1 should throw NullPointerException", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("VT1 threw (NullPointerException)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT1 wrong exception", false);
}
// VT2: min > max -> BadSequence
System.out.println(" VT2");
try {
    MullerSolver2 solver = new MullerSolver2();
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0;
    solver.solve(100, f, 5.0, -2.0);
    check("VT2 should throw BadSequence", false);
} catch (IllegalArgumentException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("VT2 threw (BadSequence)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT2 wrong exception", false);
}
// VT3: endpoint root at min
runCase("VT3", x -> x + 2.0, -2.0, 3.0, 200); // root at -2
// VT4: endpoint root at max
runCase("VT4", x -> x - 4.0, 2.0, 4.0, 200); // root at 4
// VT5: start is a root -> short-circuit
runCaseWithStart("VT5", x -> (x - 1.0) * (x - 3.0), 0.0, 5.0, 1.0, 20
0);
// VT6: start in interval & yStart*min < 0 -> progress from left side
runCaseWithStart("VT6", x -> (x - 1.0) * (x - 3.0), 0.5, 1.3, 1.2, 30
0); // expect ~1
// VT7: start in interval & yStart*max < 0 -> progress from right side
runCaseWithStart("VT7", x -> (x - 1.0) * (x - 3.0), 1.6, 3.5, 2.8, 30
0); // expect ~3
// VT8: regular convergence (bracketing not required in 2, but we sti
ll bracket)
runCase("VT8", x -> (x - 2.0) * (x - 5.0) * (x + 1.0), 1.5, 2.6, 400)
; // expect ~2
// VT9: encourage complex-
step modeling (steep exponential) -> project-to-real
// acceptance path

```

```

runCaseWithStart("VT9", x -> Math.exp(600.0 * x) - 1.0e120, -
1.0, 1.0, 0.0, 400); // root ~ ln(1e120)/600 ≈
// 0.4603
// VT10: predicted step likely outside [min,max] → safeguarded in-
range move (no
// bisection fallback in v2)
runCaseWithStart("VT10", x -> Math.expm1(120.0 * x) - 1e-8, -
0.05, 0.02, -0.04, 300); // drive large
// extrapolation
// pressure
// VT11: termination by tiny step ( $|\Delta x| \leq \text{tol}$ ) rather than  $|f| \leq f\text{Acc}$ 
runCaseWithStart("VT11", x -> (x - 1e-12) * (x - 1.0), 0.0, 2.0, 1e-
6, 400); // near-flat region around 0
// VT12: TooManyEvaluations (very small budget) on bracketed root
System.out.println("VT12");
try {
    MullerSolver2 solver = new MullerSolver2();
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.cos(x) - x; // root ~0.739, in
// [0,1]
    solver.solve(1, f, 0.0, 1.0, 0.5); // absurdly small budget
    check("VT12 should throw TooManyEvaluations", false);
} catch (IllegalStateException ok) {
    System.out.println("threw: " + ok.getClass().getSimpleName());
    check("VT12 threw (TooManyEvaluations)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT12 wrong exception", false);
}
// --- VT13: force delta < 0 (complex-step branch) ---
// Very steep exponential with symmetric bracket and start at 0 to en-
courage a
// negative discriminant
// in the quadratic model (complex step -> project-to-real path).
runCaseWithStart("VT13", x -> Math.exp(2000.0 * x) - 1.0e300, // huge
K, super steep
-1.0, 1.0, 0.0, 500);
// --- VT14: force denominator == 0 (a=0, b=0) ---
// Constant nonzero function: all divided differences 0 => a=0, b=0,
delta=0.
// Typical Muller2 implementations hit the denominator==0 safeguard b-
ranch here.
runCase("VT14", x -> 1.0, // no real root; branch behavior (safeguard
/perturb) is what we want to execute

```

```

    -2.0, 2.0, 100);
    // --
- VT15: ensure (x != x1 && x != x2) so the 'while (x==x1 || x==x2)' is
NOT
    // entered ---
    // Mild polynomial where the Muller step proposes a genuinely new poi
nt distinct
    // from both previous points.
    runCaseWithStart("VT15", x -> (x - 0.5) * (x + 1.0) * (x - 2.0), // r
oots at -1, 0.5, 2
    -0.2, 1.2, 0.1, 400); // bracket around 0.5; start off-
center to avoid degeneracy
    // --- VT16: denominator == 0 → hit the rare "random jump" branch
    // Constant nonzero function ⇒ all divided differences zero ⇒ a=0, b
=0, delta=0
    // ⇒ denominator==0.
    runCaseWithStart("VT16", x -> 42.0, // strictly constant; not zero to
avoid trivial root short-circuit
    -
    3.0, 7.0, 1.2345, // wide interval + explicit start to enter the Muller
step path
    50);
//VT17
    runCaseWithStart("VT17", x -> (x < 0.0 ? 7.0 : 7.0), // exactly const
ant
    -2.0, 2.0, 0.5, 50);
//VT18
    runCaseWithStart("VT18", x -> 8.0, // power of two keeps all ops exac
t in IEEE-754
    -4.0, 4.0, 0.0, 50);
// VT19
    {
        double min = -1.0;
        double start = 0.2;
        double max = 2.5;
        double C = 42.0; // nonzero to avoid endpoint short-circuit
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - min) * (x - start) * (x - max)
        + C;
        runCaseWithStart("VT19", f, min, max, start, 200);
    }
//VT20
    runCaseWithStart("VT20", x -> Math.exp(2000.0 * x) - 1.0e300, // huge
K, super steep

```

```

        -1.0, 1.0, 0.0, 500);
    System.out.println("=== Done MullerSolver2 V-method tests ===");
}
}
//FT
class MS2FT {
    private static void runCase(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");
        try {
            MullerSolver2 solver = new MullerSolver2();
            double r = solver.solve(maxEval, f, min, max);
            System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
        }
    }
    private static void runCaseWithStart(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f,
        double min, double max, double start, int maxEval) {
        System.out.println("---- " + name + " (with start) ----");
        try {
            MullerSolver2 solver = new MullerSolver2();
            double r = solver.solve(maxEval, f, min, max, start);
            System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
        }
    }
    // Helpers to make messy random doubles
    private static double rSigned(Random rnd) {
        // spread across magnitudes, but not too huge to avoid NaN in exp-heavy
        // functions
        double s = rnd.nextBoolean() ? 1 : -1;
        double base = Math.pow(10, rnd.nextDouble() * 4.0 - 2.0); // 10^-2 .. 10^2
        return s * base * (0.5 + rnd.nextDouble() * 999.5); // ~[5e-3..1e5] with sign
    }
    private static double rInRange(Random rnd, double lo, double hi) {

```

```

    return lo + rnd.nextDouble() * (hi - lo);
}
// Random polynomial (degree 1..5) with random coefficients (ascending
powers)
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randPoly(Random rnd) {
    int deg = 1 + rnd.nextInt(5);
    final double[] c = new double[deg + 1];
    for (int i = 0; i < c.length; i++) {
        // Mix tiny and big-ish coefficients
        double m = rnd.nextBoolean() ? Math.pow(10, -
rnd.nextInt(12)) : Math.pow(10, rnd.nextInt(4));
        c[i] = (rnd.nextBoolean() ? 1 : -1) * m * rnd.nextDouble();
    }
    return x -> {
        double v = 0.0;
        double p = 1.0;
        for (double ci : c) {
            v += ci * p;
            p *= x;
        }
        return v;
    };
}
// Random "safe" exponential form: exp(k*x) - offset, with bounded k/o
ffset to
// avoid overflow
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randExp(Random rnd) {
    double k = rInRange(rnd, 5.0, 300.0) * (rnd.nextBoolean() ? 1 : -1);
    double off = Math.pow(10, rInRange(rnd, -3.0, 8.0)); // 1e-3 .. 1e8
    return x -> Math.exp(Math.max(-
700.0, Math.min(700.0, k * x))) - off; // clamp exponent
}
// Trig cocktail: A*sin(ax + b) + B*cos(cx + d) + tilt*x + bias
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randTrig(Random rnd) {
    double A = rInRange(rnd, -5, 5);
    double a = rInRange(rnd, 0.1, 50);
    double b = rInRange(rnd, -5, 5);
    double B = rInRange(rnd, -5, 5);
    double c = rInRange(rnd, 0.1, 50);
    double d = rInRange(rnd, -5, 5);
    double tilt = rInRange(rnd, -1, 1);

```

```

    double bias = rInRange(rnd, -1, 1);
    return x -> A * Math.sin(a * x + b) + B * Math.cos(c * x + d) + tilt
* x + bias;
}
// Rational-ish but safe:  $x - p / (1 + q x^2)$ 
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randRational(Random rnd) {
    double p = rSigned(rnd);
    double q = Math.abs(rSigned(rnd)) + 1e-9;
    return x -> x - p / (1.0 + q * x * x);
}
// Logistic-ish / tanh-ish
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randSigmoid(Random rnd) {
    double k = rInRange(rnd, 0.5, 50.0) * (rnd.nextBoolean() ? 1 : -1);
    double shift = rInRange(rnd, -2.0, 2.0);
    double bias = rInRange(rnd, -1.0, 1.0);
    return x -> 1.0 / (1.0 + Math.exp(-k * (x - shift))) - 0.5 + bias;
}
// Pick a random function family
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion randomFunction(Random rnd) {
    int t = rnd.nextInt(5);
    switch (t) {
    case 0:
        return randPoly(rnd);
    case 1:
        return randExp(rnd);
    case 2:
        return randTrig(rnd);
    case 3:
        return randRational(rnd);
    default:
        return randSigmoid(rnd);
    }
}
public void Ftest() {
    // Fixed seed → reproducible random fuzz. Change to get a new batch.
    long seed = 0xC0FFEE_F00D_BA5EL ^ System.nanoTime();
    Random rnd = new Random(seed);
    System.out.println("=== MullerSolver2 RANDOM FUZZ: 20 cases ===");
    System.out.printf("seed=0x%016X\n", seed);
    for (int i = 1; i <= 20; i++) {
        // Random interval construction

```

```

double a = rInRange(rnd, -20.0, 20.0);
double b = rInRange(rnd, -20.0, 20.0);
double min = Math.min(a, b);
double max = Math.max(a, b);
if (min == max)
    max = min + 1.0; // avoid zero-length interval
// With small probability, flip to make min>max (to probe argument c
hecks)
if (rnd.nextDouble() < 0.1) {
    double tmp = min;
    min = max;
    max = tmp;
}
int maxEval = 100 + rnd.nextInt(900); // [100, 999]
org.apache.commons.math4.legacy.analysis.UnivariateFunction f = rand
omFunction(rnd);
// 50% cases with start; start either inside or slightly outside the
range
boolean withStart = rnd.nextBoolean();
if (withStart) {
    double start;
    if (rnd.nextBoolean()) {
        start = rInRange(rnd, Math.min(min, max), Math.max(min, max)); //
inside
    } else {
        // slightly outside
        double span = Math.abs(max - min);
        start = (rnd.nextBoolean() ? min - 0.1 * span : max + 0.1 * span);
    }
    runCaseWithStart(String.format("RFZ%02d", i), f, min, max, start, m
axEval);
} else {
    runCase(String.format("RFZ%02d", i), f, min, max, maxEval);
}
}
System.out.println("=== Done RANDOM FUZZ ===");
}
}
//Z3
class MS2Z3 {
    private static void runCase(String name, org.apache.commons.math4.lega
cy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");

```

```

try {
    MullerSolver2 solver = new MullerSolver2();
    double r = solver.solve(maxEval, f, min, max);
    System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
}
}

private static void runCaseWithStart(String name, org.apache.commons.m
ath4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        MullerSolver solver = new MullerSolver();
        double r = solver.solve(maxEval, f, min, max, start);
        System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

// Build a linear function that hits f(min)=yMin and f(max)=yMax.
// If min==max, fallback to constant yMin.
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion linearThrough(double min, double yMin,
    double max, double yMax) {
    if (Double.compare(min, max) == 0) {
        final double c = yMin;
        return x -> c;
    }
    final double a = (yMax - yMin) / (max - min);
    final double b = yMin - a * min;
    return x -> a * x + b;
}

public void Z3test() {
    System.out.println("=== MullerSolver (Z3-derived) : 20 cases ===");
    // Row 1: f=null -> NullArgument (min=0, max=0)
    runCase("Z3-01 f=null -> NullArgument", null, 0.0, 0.0, 50);
    // Row 2: f=null -> NullArgument (min=-1, max=-1)
    runCase("Z3-02 f=null -> NullArgument", null, -1.0, -1.0, 50);
    // Row 3: min>max -> BadSequence (max=0, min=1/2)
    runCase("Z3-03 min>max -> BadSequence", x -> x - 1.0, 0.5, 0.0, 100);
    // Row 4: min>max -> BadSequence (max=-1/4, min=0)

```

```

runCase("Z3-04 min>max -> BadSequence", x -> x + 0.25, 0.0, -
0.25, 100);
// Row 5: not bracketed -> NoBracketing (yMin=0 at min=-999, yMax=-
1 at max=0)
runCase("Z3-05 not bracketed -> NoBracketing", linearThrough(-
999.0, 0.0, 0.0, -1.0), -999.0, 0.0, 200);
// Row 6: not bracketed -> NoBracketing (yMin=-499 at min=-
999, yMax=0 at
// max=-999.5)
runCase("Z3-06 not bracketed -> NoBracketing", linearThrough(-
999.0, -499.0, -999.5, 0.0), -999.0, -999.5, 200);
// Row 7: yMin=0 -> r=min (min=-1, max=0)
runCase("Z3-07 yMin=0 -> r=min", x -> x - (-1.0), -1.0, 0.0, 200);
// Row 8: yMin=0 -> r=min (min=-2, max=0)
runCase("Z3-08 yMin=0 -> r=min", x -> x - (-2.0), -2.0, 0.0, 200);
// Row 9: yMax=0 -> r=max (min=0, max=1)
runCase("Z3-09 yMax=0 -> r=max", x -> x - 1.0, 0.0, 1.0, 200);
// Row 10: yMax=0 -> r=max (min=-999.5, max=-999)
runCase("Z3-10 yMax=0 -> r=max", x -> x - (-999.0), -999.5, -
999.0, 200);
// Row 11: start in interval & yStart=0 -> r=start (min=0,max=1,start
=0)
runCaseWithStart("Z3-
11 start in interval & yStart=0 -> r=start", x -> x - 0.0, 0.0, 1.0, 0.
0, 200);
// Row 12: start in interval & yStart=0 -> r=start (min=-
999.5,max=0,start=-999)
runCaseWithStart("Z3-
12 start in interval & yStart=0 -> r=start", x -> x - (-999.0), -
999.5, 0.0, -999.0, 200);
// Row 13: start*min opposite sign -> r∈(min,start] (min=-
999,start=0, choose
// max=1)
runCaseWithStart("Z3-13 start*min opposite sign", x -> x + 1.0, -
999.0, 1.0, 0.0, 200);
// Row 14: start*min opposite sign -> r∈(min,start] (min=-
999.5,start=-999,
// choose max=-998.5)
runCaseWithStart("Z3-14 start*min opposite sign", x -> x + 999.25, -
999.5, -998.5, -999.0, 200);
// Row 15: start*max opposite sign -> r∈[start,max) (max=1,start=0, c
hoose
// min=-1)

```

```

    runCaseWithStart("Z3-15 start*max opposite sign", x -> x - 0.2, -
1.0, 1.0, 0.0, 200);
    // Row 16: start*max opposite sign ->  $r \in [start, max)$  ( $max=-999, start=-999.5,$ 
    // choose  $min=-1000$ )
    runCaseWithStart("Z3-16 start*max opposite sign", x -> x + 999.25, -
1000.0, -999.0, -999.5, 200);
    // Row 17:  $\exists x_{New}$ : in interval & ( $|f| \leq f_{Acc}$  or  $|\Delta| \leq tol$ ) ->  $r=x_{New}$ 
    // ( $min=-1, max=0, start=0, x_{New}=0, x_{Prev}=0$ ) - simple root at 0
    runCaseWithStart("Z3-17 exists xNew ->  $r=x_{New}$ ", x -> x, -
1.0, 0.0, 0.0, 200);
    // Row 18:  $\exists x_{New}$ : in interval & ... ->  $r=x_{New}$ 
    // ( $min=0, max=1, start=-1, x_{New}=1, x_{Prev}=-1$ )
    runCaseWithStart("Z3-
18 exists xNew ->  $r=x_{New}$ ", x -> (x) * (x - 1.0), 0.0, 1.0, -1.0, 200);
    // Row 19: quadraticStepInvalid -> pick xMid and  $r=x_{Mid}$  ( $min=-1, max=0, x_{Mid}=-0.5$ )
    // Use a very steep exponential to encourage Muller fallback behavior
    .
    runCase("Z3-
19 quadraticStepInvalid -> pick xMid", x -> Math.exp(200.0 * x) - 1e-
20, -1.0, 0.0, 300);
    // Row 20: evaluations>maxEval -> TooManyEvaluations
    // (evaluations=1, maxEval=4, max=0, min=0)
    // We still just run it as a concrete call with your bounds.
    runCase("Z3-
20 evaluations>maxEval -> TooManyEvaluations", x -> x - 0.1, 0.0, 0.0,
4);
    System.out.println("=== Done ===");
}
}
class MS2test {
    public static void main(String[] args) {
        // VT
        MS2VT t1 = new MS2VT();
        t1.Vtest();
        // FT
        MS2FT t2 = new MS2FT();
        //      t2.Ftest();
        // Z3
        MS2Z3 t3 = new MS2Z3();
        //      t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 95%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver2.java	31.9 %	22	47	69
> MS2FT	0.0 %	0	31	31
> MS2Z3	0.0 %	0	2	2
> MS2VT	18.8 %	3	13	16
▼ MullerSolver2	95.0 %	19	1	20
◇ doSolve()	95.0 %	19	1	20
● MullerSolver2()		0	0	0
● MullerSolver2(double)		0	0	0
> MS2test		0	0	0

3. V-Method testing instruction coverage: 94.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MullerSolver2.java	38.8 %	702	1,106	1,808
> MS2FT	0.5 %	3	590	593
> MS2Z3	0.8 %	3	364	367
> MS2VT	76.6 %	438	134	572
> MS2test	83.3 %	15	3	18
▼ MullerSolver2	94.2 %	243	15	258
◇ doSolve()	95.9 %	235	10	245
● MullerSolver2()	100.0 %	4	0	4
● MullerSolver2(double)	100.0 %	4	0	4

4. Fuzz testing branch coverage: 15%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver2.java	5.8 %	4	65	69
> MS2FT	0.0 %	0	31	31
> MS2VT	0.0 %	0	16	16
▼ MullerSolver2	15.0 %	3	17	20
◇ doSolve()	15.0 %	3	17	20
● MullerSolver2()		0	0	0
● MullerSolver2(double)		0	0	0
> MS2Z3	50.0 %	1	1	2
> MS2test		0	0	0

5. Fuzz testing instruction coverage: 20.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MullerSolver2.java	22.7 %	410	1,398	1,808
> MS2FT	0.5 %	3	590	593
> MS2VT	0.5 %	3	569	572
▼ MullerSolver2	20.5 %	53	205	258
◇ doSolve()	18.4 %	45	200	245
● MullerSolver2()	100.0 %	4	0	4
● MullerSolver2(double)	100.0 %	4	0	4
> MS2test	83.3 %	15	3	18
> MS2Z3	91.6 %	336	31	367

6. Z3 branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver2.java	60.9 %	42	27	69
> MS2VT	0.0 %	0	16	16
> MS2Z3	0.0 %	0	2	2
▼ MullerSolver2	75.0 %	15	5	20
◇ doSolve()	75.0 %	15	5	20
● MullerSolver2()		0	0	0
● MullerSolver2(double)		0	0	0
> MS2FT	87.1 %	27	4	31
> MS2test		0	0	0

7. Z3 instruction coverage: 92.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  MullerSolver2.java	 44.9 %	812	996	1,808
>  MS2VT	 0.5 %	3	569	572
>  MS2Z3	 0.8 %	3	364	367
>  MS2test	 83.3 %	15	3	18
▼  MullerSolver2	 92.6 %	239	19	258
 doSolve()	 94.3 %	231	14	245
 MullerSolver2()	 100.0 %	4	0	4
 MullerSolver2(double)	 100.0 %	4	0	4
>  MS2FT	 93.1 %	552	41	593